УНИВЕРСИТЕТ ИТМО

Дисциплина: Backend

ОТЧЁТ
по лабораторной работе № 2

«Микросервисная архитектура приложения
для бронирования столиков в ресторанах»

Выполнила: Софья Кисель

Группа: БР 1.1

Санкт-Петербург, 2026

1. Цель и постановка задачи

Цель лабораторной работы — изучить принципы микросервисной архитектуры и разделить серверное приложение для бронирования столиков в ресторанах на самостоятельные сервисы. Каждый сервис отвечает за отдельную предметную область и предоставляет HTTP API для работы с соответствующими данными.

2. Структура и архитектура приложения

В проекте выделены пять сервисов: Auth Service, Restaurant Service, Menu Service, Reservation Service и Review Service. Такое разделение позволяет отделить аутентификацию, работу с ресторанами и меню, бронирования и отзывы друг от друга.

Сервисы реализованы на Node.js с использованием TypeScript и Express. Для взаимодействия с PostgreSQL применяется TypeORM. Состояние сервиса проверяется через health-check маршруты.

3. Микросервисы и их ответственность

Сервис	Ответственность
Auth Service	Регистрация и аутентификация пользователей; выдача JWT-токена.
Restaurant Service	Данные ресторанов и связанные сущности, включая кухни и столики.
Menu Service	Меню ресторана и позиции меню.
Reservation Service	Данные бронирований: пользователь, столик, дата и время, число гостей и статус.
Review Service	Данные отзывов: пользователь, ресторан, оценка, комментарий и временные отметки.

4. Организация хранения данных

Для сервисов предусмотрено раздельное хранение данных в PostgreSQL. Сущности каждого сервиса описываются TypeORM-моделями. Связи с объектами, принадлежащими другим сервисам, представлены внешними идентификаторами, например user_id или restaurant_id, а не общей ORM-связью через единую базу данных.

Такой подход разделяет ответственность за данные. При этом полноценная согласованность между сервисами зависит от реализованных межсервисных запросов или событийных механизмов.

5. Реализация HTTP API

Сервисы предоставляют HTTP-маршруты для проверки состояния и получения данных. В текущем состоянии реализации доступны health-check endpoint-ы и GET-операции для получения списков или данных соответствующих ресурсов. Например, Reservation Service предоставляет получение списка бронирований, Menu Service — меню выбранного ресторана, а Restaurant Service — список ресторанов.

В некоторых сервисах API пока ограничено чтением данных. Поэтому создание, изменение и удаление записей не следует считать реализованными для всех предметных областей.

6. Запуск и проверка работоспособности

Для проверки сервисы запускались отдельно, а запросы направлялись к их HTTP endpoint-ам. Также использовались контейнеры PostgreSQL, описанные в Docker Compose. Проверялась доступность health-check маршрутов и ответов GET-запросов.

Результаты проверок отдельных сервисов

Компонент	Проверка	Результат
Restaurant Service	Health-check и GET списка ресторанов	Сервис отвечает; список возвращается
Menu Service	Health-check и GET меню ресторана	Сервис отвечает; при отсутствии данных — пустой список
Reservation Service	Health-check и GET списка бронирований	Сервис отвечает; при отсутствии данных — пустой список
Review Service	Health-check и GET списка отзывов	Сервис отвечает; при отсутствии данных — пустой список
Auth Service	Регистрация тестового пользователя	Успешный ответ; пароль хранится в виде хэша, возвращается JWT

7. Вывод

В результате лабораторной работы серверное приложение для бронирования столиков в ресторанах было декомпозировано на микросервисы по основным предметным областям. Для сервисов определены зоны ответственности, модели данных и HTTP API. Выполнена проверка доступности отдельных компонентов и их базовых операций. Реализация демонстрирует базовые принципы микросервисной архитектуры и может быть расширена межсервисным взаимодействием и дополнительными операциями API.

Репозиторий проекта: https://github.com/teneramite/ITMO-ACS-Backend-2026-A
Pull Request: будет указан после публикации ЛР2.